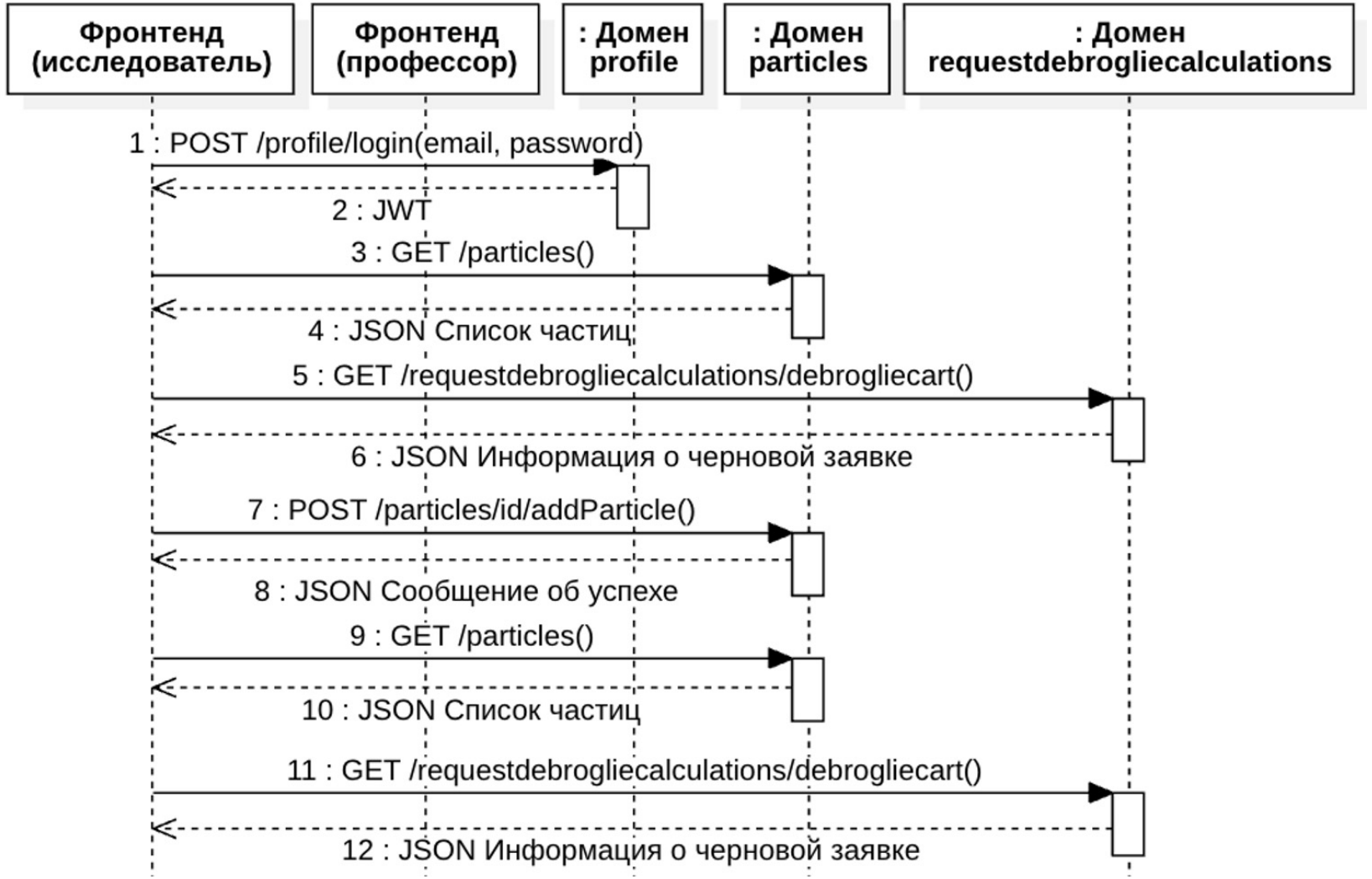
Лекция 7
JWT. SSO

Проектирование систем и продуктовая веб-разработка

Канев Антон Игоревич

Диаграмма последовательности

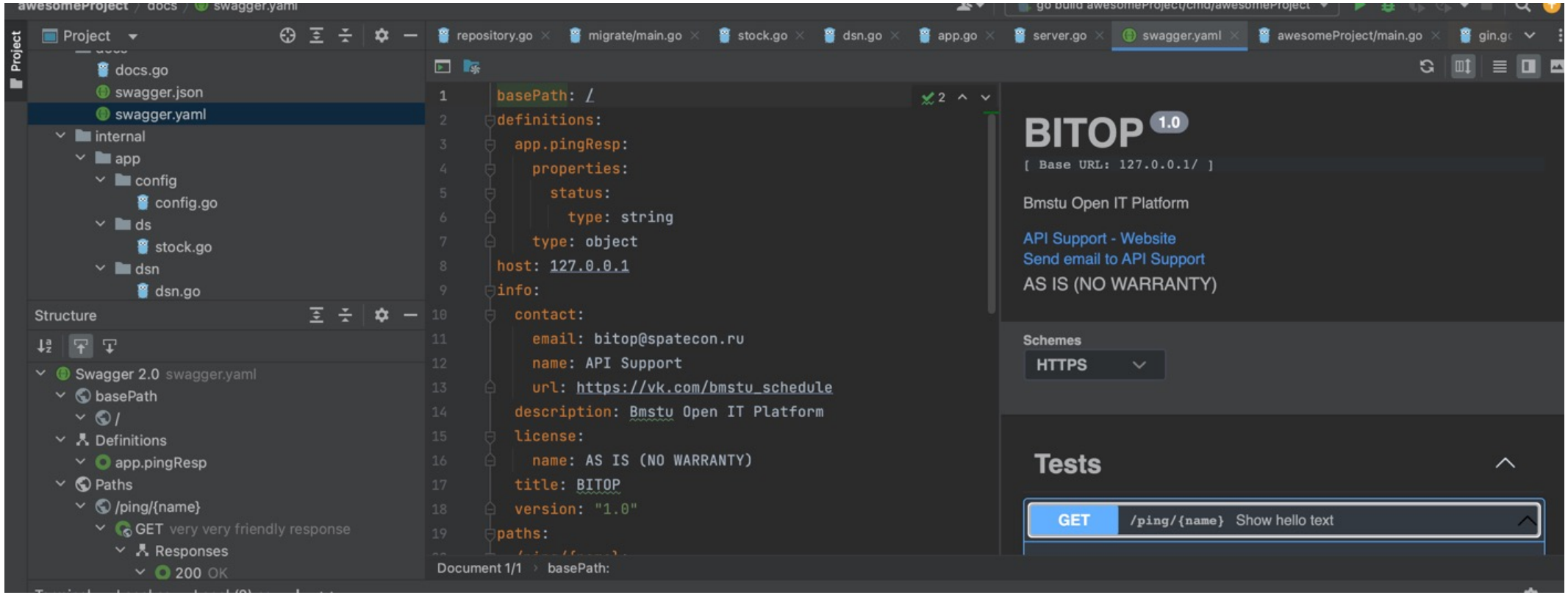
- Этот пример плохой - **Слишком мелкая диаграмма** для отчета
- На этой диаграмме мы хотим описать последовательность действий
- Например HTTP запросы, которые нам нужно выполнить для «ура сценария», например, купить товар.
- Эта последовательность нам пригодится при тестировании сервиса
- Мы укажем какие действ в каком порядке надо выполнить и какие данны мы получаем в результате



Swagger Go

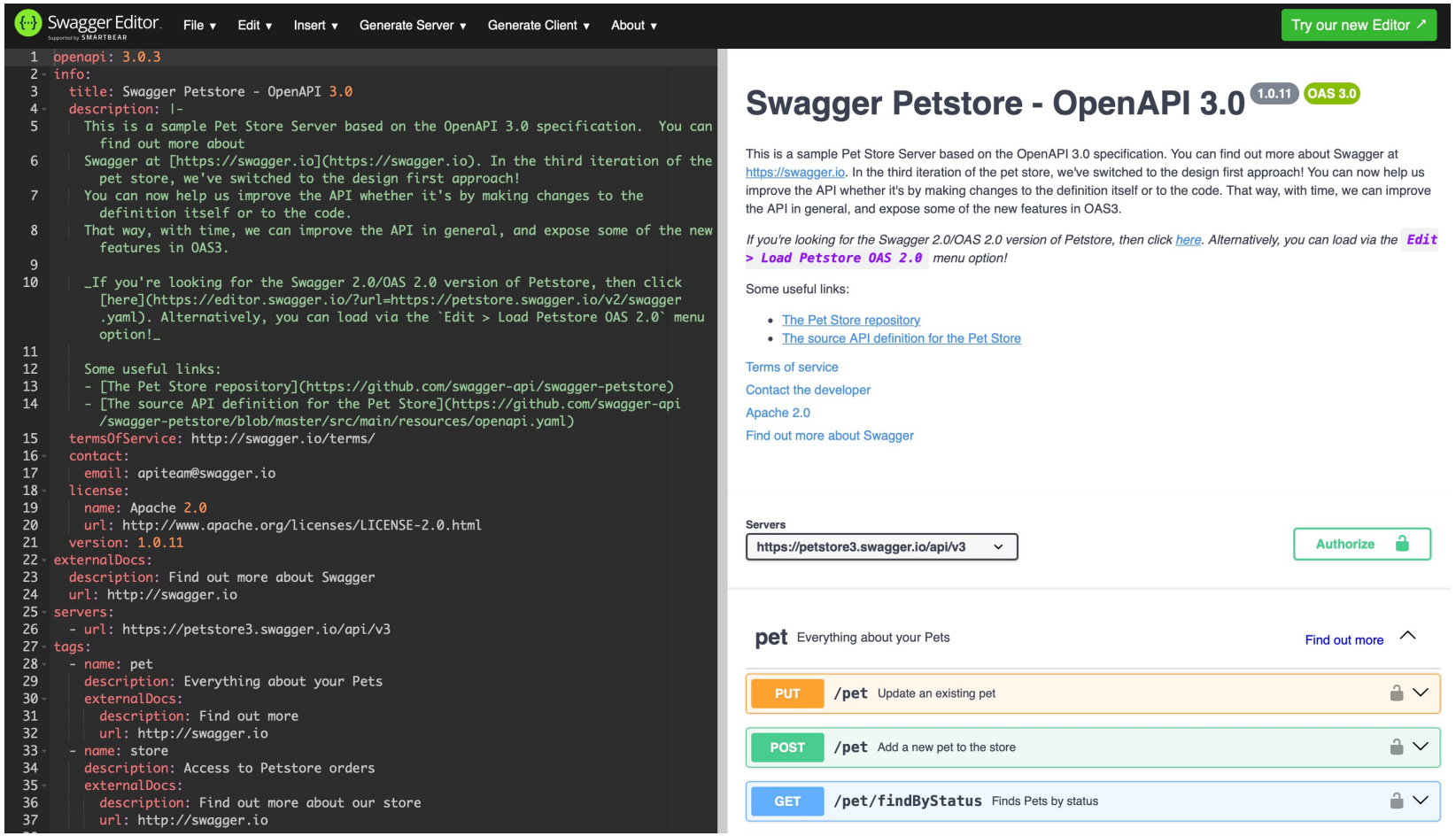
- Опишем наши методы в коде
- Сгенерируем файлы документации

```
// Ping godoc
// @Summary      Show hello text
// @Description  very very friendly response
// @Tags         Tests
// @Produce      json
// @Success      200  {object}  pingResp
// @Router       /ping/{name} [get]
func (a *Application) Ping(gCtx *gin.Context) {
   name := gCtx.Param("name")
   gCtx.String(http.StatusOK, "Hello %s", name)
}
```



Swagger editor (для курсовой)

- Описанные файлы swagger можно легко просматривать, редактировать с помощью различных сервисов
- Например, swagger editor – позволяет посмотреть в графическом виде загруженный файл



https://editor.swagger.io

Аутентификация, Авторизация (Повтор)

Аутентифика́ция (*authentication*) — процедура проверки подлинности, например:

- проверка подлинности пользователя путём сравнения введённого им пароля (для указанного логина) с паролем, сохранённым в базе данных пользовательских логинов;
- подтверждение подлинности электронного письма путём проверки цифровой подписи письма по открытому ключу отправителя;

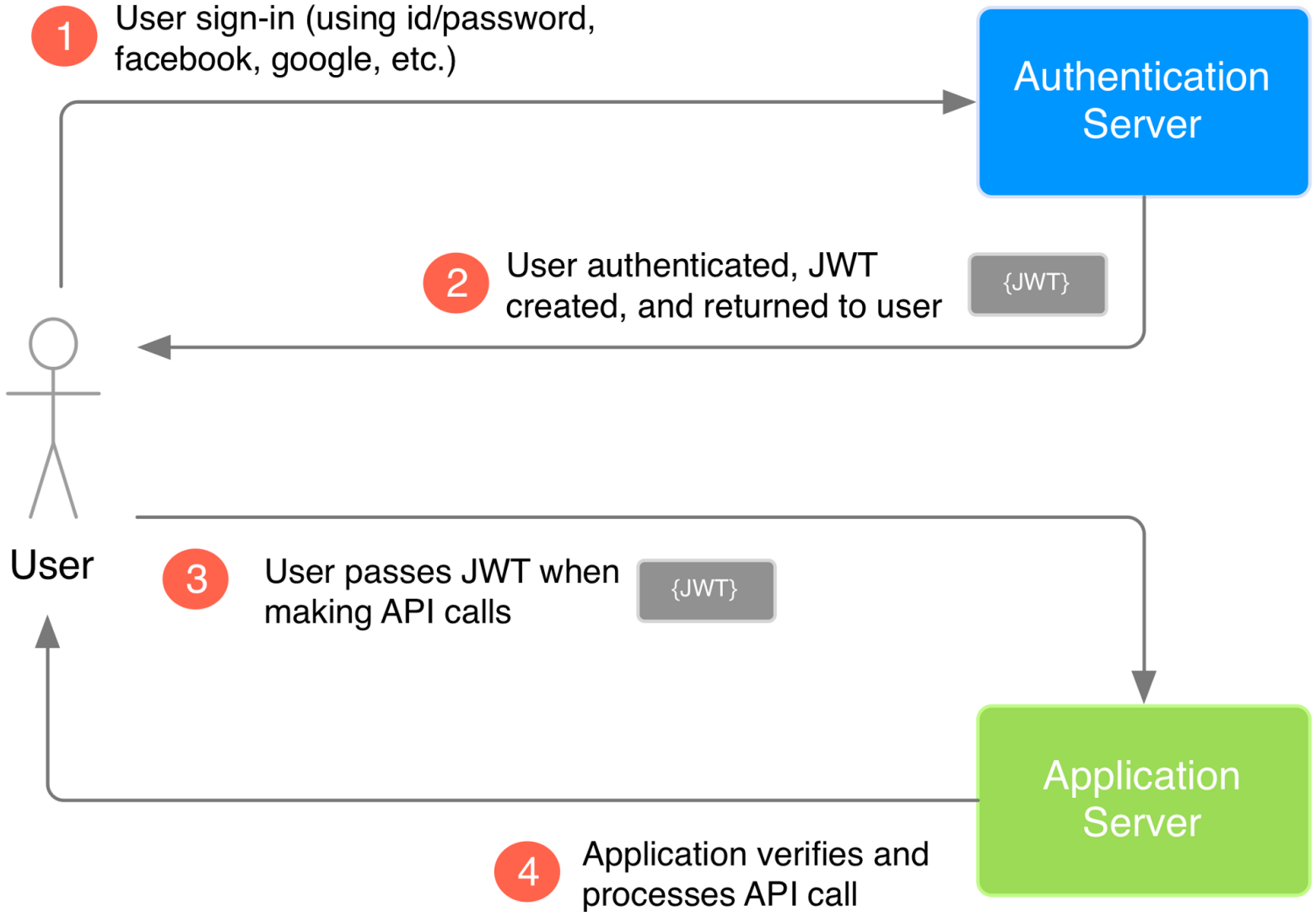
Идентификация — процедура, в результате выполнения которой для субъекта идентификации выявляется его идентификатор, однозначно определяющий этого субъекта в информационной системе (**Определяем** пользователя для заявки).

Авториза́ция (*authorization* «разрешение; уполномочивание») — предоставление определённому лицу или группе лиц прав на выполнение определённых действий; а также процесс проверки (подтверждения) данных прав при попытке выполнения этих действий (**гость** смотрит товар, **покупатель** добавляет в заказ, **продавец** - редактирует).

- Авторизация производит контроль доступа к различным ресурсам системы в процессе работы легальных пользователей после успешного прохождения ими аутентификации.

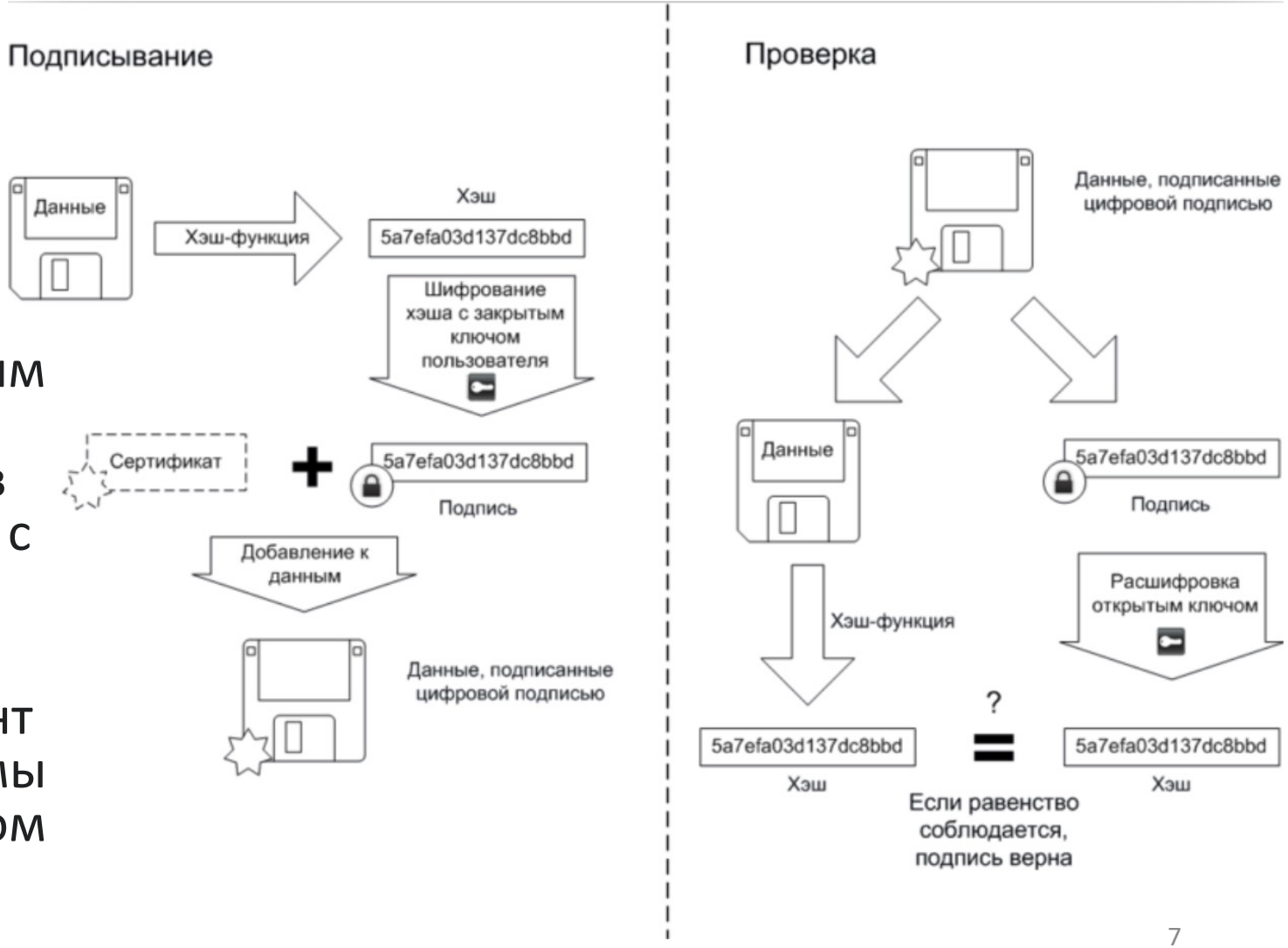
JWT

- **JSON Web Token**
- Как правило, используется для передачи данных для аутентификации в клиент-серверных приложениях.
- Токены создаются сервером, **подписываются** секретным ключом (цифровая подпись) и передаются клиенту, который в дальнейшем использует данный токен для подтверждения подлинности аккаунта.



Цифровая подпись *

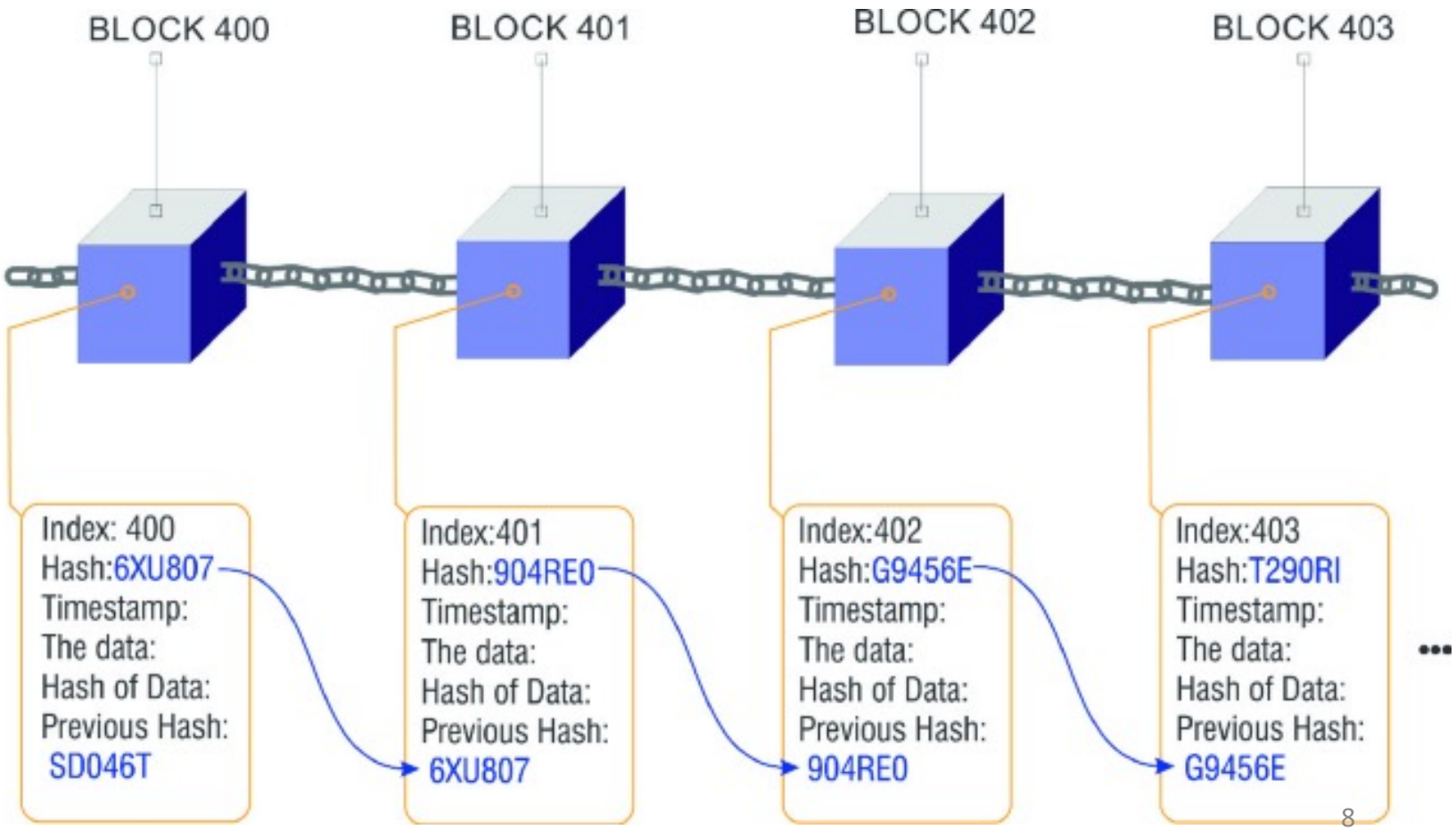
- **Электро́нная цифровая по́дпись** позволяет подтвердить авторство электронного документа
- Подпись связана как с автором (ключ), так и с самим документом с помощью криптографических методов и не может быть подделана с помощью обычного копирования.
- Автор подписывает документ (хэш) закрытым ключом, а мы проверяем открытым ключом (который есть у всех)



Blockchain

- Последовательность действий/операций
- В каждом блоке новые действия

- Каждый следующий блок подписывает ссылку на предыдущий
- Подписываем «историю» действий



RSA *

- Первый алгоритм одновременно для ассиметричного шифрования и цифровой подписи
- Для цифровой подписи просто меняем местами
- Открытая экспонента ***e*** и модуль ***n*** публикуются
- Экспонента ***d*** и модуль ***n*** это закрытый ключ автора

$(kx + 1) \bmod x$ = ?

$(kx + 1)^e \bmod x$ = ?

$\big((kx + b) \bmod x\big)^e \bmod x$ = ?

Шифрование:

- Генерируем два простых числа $n = p \cdot q$
- Считаем для них функцию Эйлера $\varphi(n) = (p-1) \cdot (q-1)\,;$
- Выбираем ***d*** и ***e*** взаимно простые с значением функции Эйлера $d \cdot e \equiv 1 \pmod{\varphi(n)}$

- Взять *открытый ключ* (e, n) Алисы
- Взять *открытый текст* m
- Зашифровать сообщение с использованием открытого ключа Алисы:

$$c = E(m) = m^e \mod n \quad (1)$$

Алгоритм расшифрования:

- Принять зашифрованное сообщение c
- Взять свой *закрытый ключ* (d, n)
- Применить закрытый ключ для расшифрования сообщения:

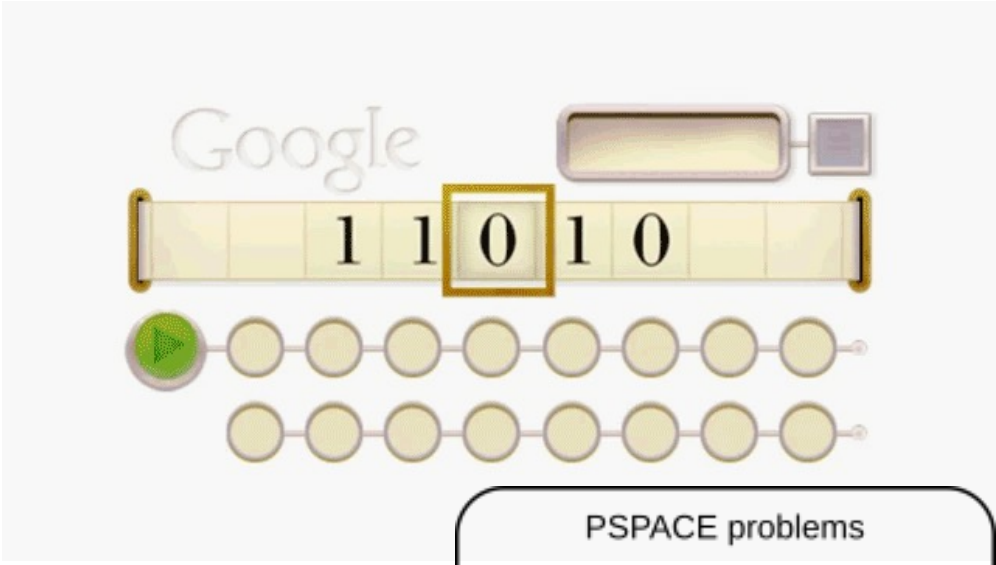
$$m = D(c) = c^d \mod n \quad (2)$$

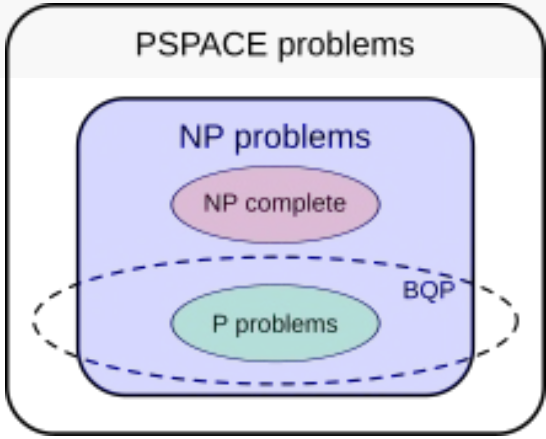
$$D\left(E\left(m\right)\right) = E\left(D\left(m\right)\right) = m^{ed} \mod n$$

Доказательство можно **проверить**: https://ru.wikipedia.org/wiki/RSA

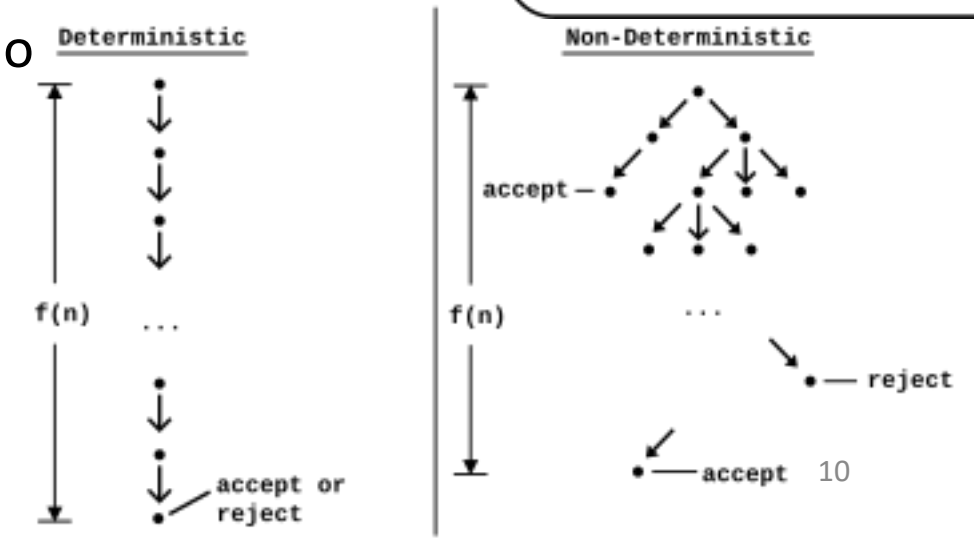
NP-трудные задачи

- В основе любого современного компьютера лежит машина Тьюринга
- Факторизация (взлом RSA) обладает экспоненциальной сложностью – растет экспоненциально от количества разрядов числа.
- Но решение проверяется (расшифровка по ключу) полиномиально - NP

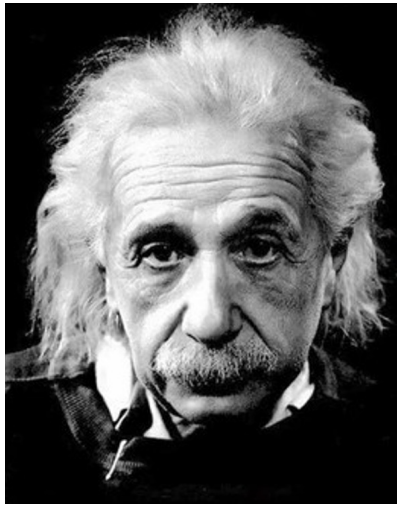




- Недерминированная машина может одновременно решать несколько вариантов, «клонироваться»
- NP-трудные задачи выполняются на недерминированной машине Тьюринга за полиномиальное время
- BQP – полиномиальные квантовые алгоритмы



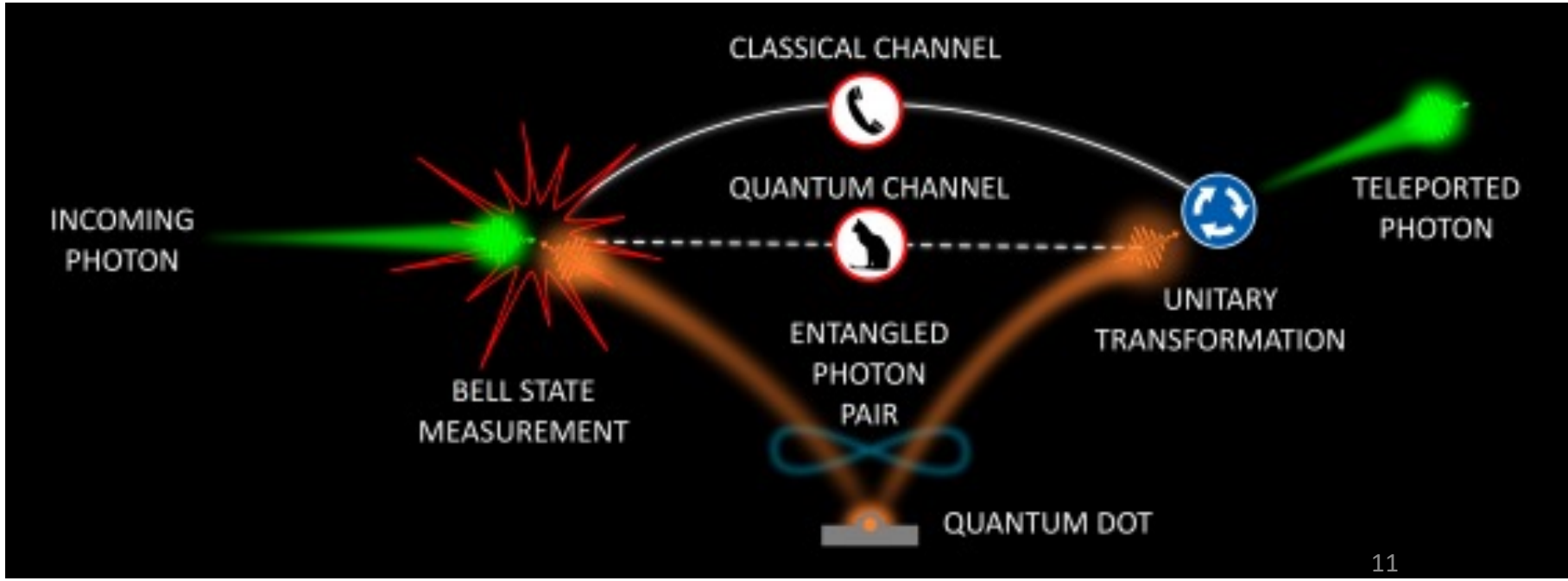
Квантовая телепортация



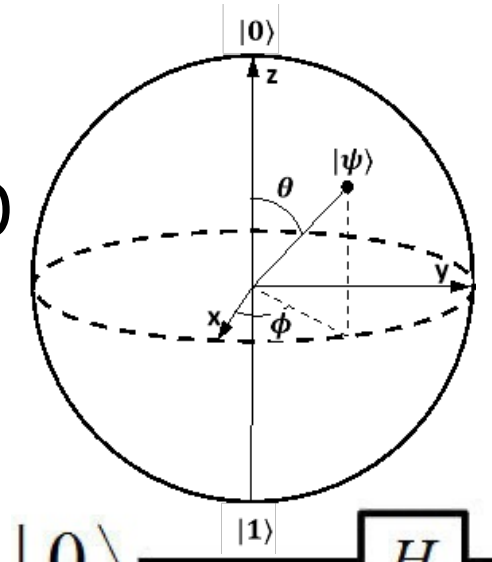


I like to think that the **Moon** is there even if I **am not looking** at it

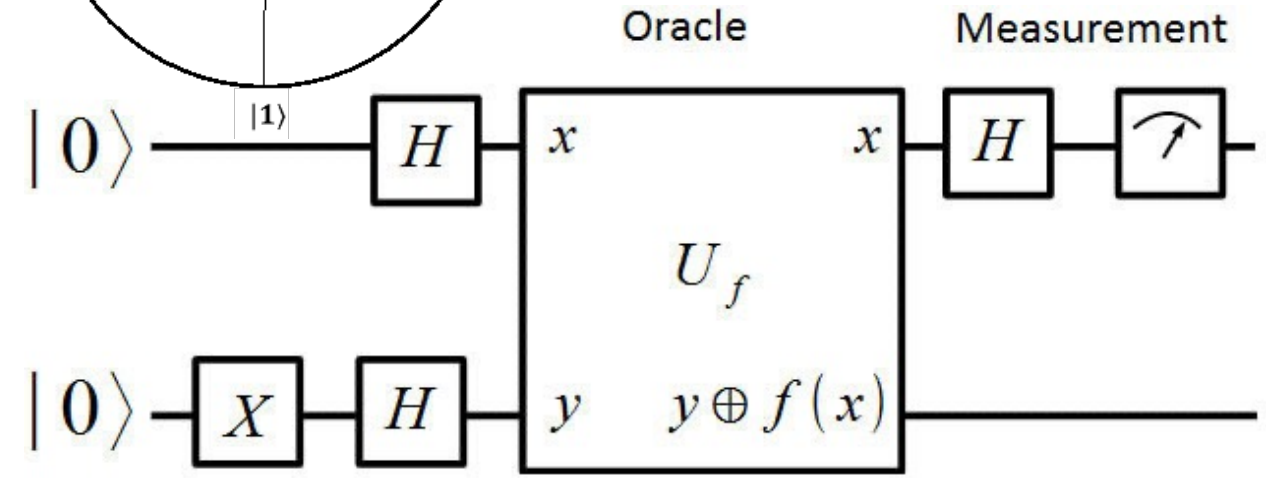
- В 1935 Эйнштейн, Розен и Подольский опубликовали статью с ЭПР-парадоксом, который говорил о мгновенной передаче информации при измерении одной частицы – значит квантовая механика не полна
- Нобелевская премия по физике 2022 –доказательство нарушения неравенств Белла. КМ полна
- Но для передачи информации нужен обычный канал – осмысленный сигнал передать быстрее света не получится
- Зато можно создать два генератора случайных чисел в разных местах



Квантовый компьютер



- Алгоритм Шора позволяет разложить число на простые множители за полиномиальное время, в 2001 запущен на квантовом компьютере IBM
- Слева пример алгоритма Дойча: два кубита в суперпозиции, а затем запутаны

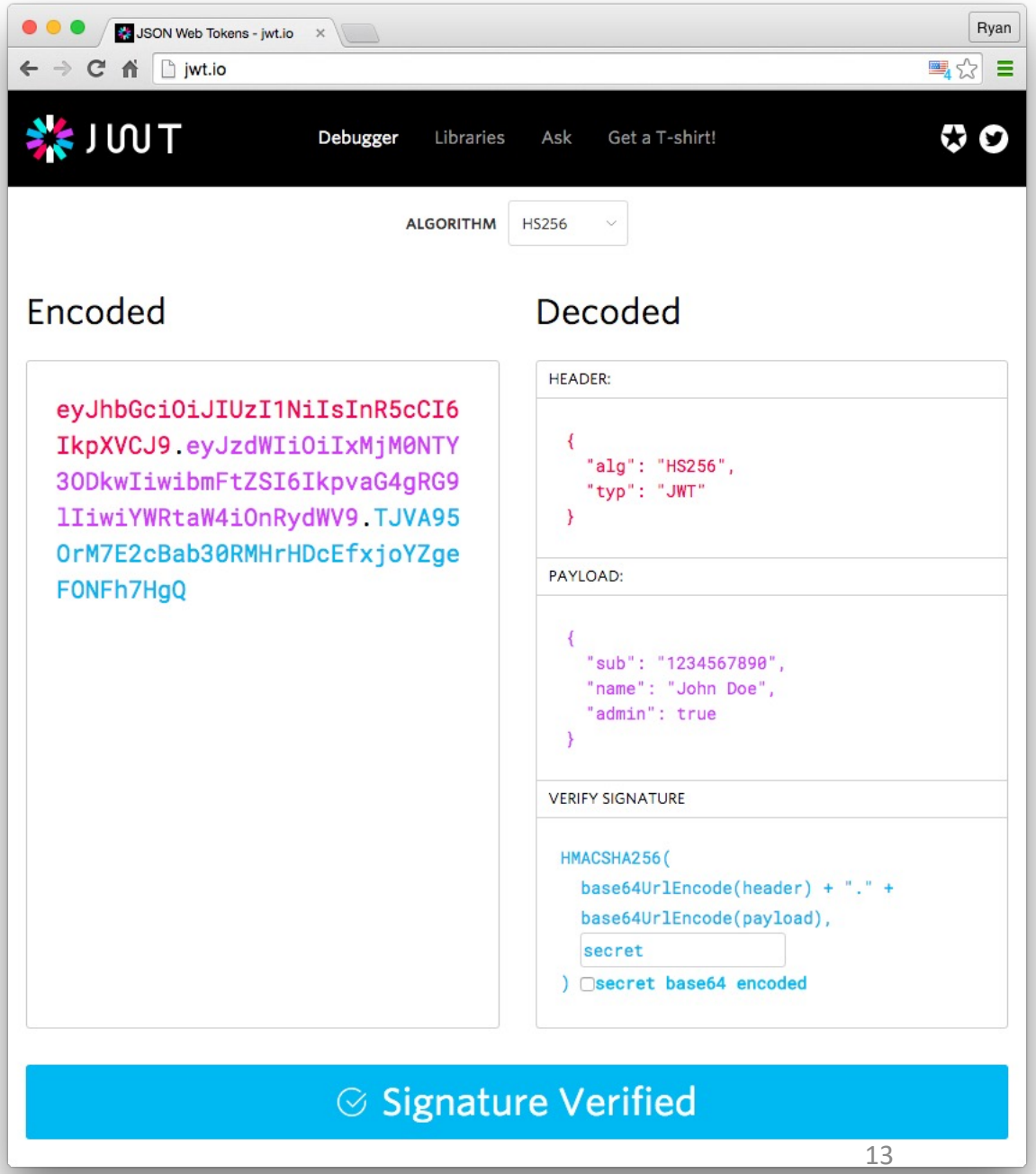




- Одной из популярных на данный момент интерпретаций КМ является many-worlds interpretation - мы видим результат **только нашего world**
- Наблюдатель как бы расщепляется на несколько версий, мы видим только один вариант – коллапс после измерения, не можем знать все результаты

JWT *

- Токен JWT состоит из трех частей: заголовка (header), полезной нагрузки (payload) и подписи или данных шифрования.
- Первые два элемента — это JSON объекты определенной структуры. Третий элемент вычисляется на основании первых и зависит от выбранного алгоритма (в случае использования неподписанного JWT может быть опущен).
- Токены могут быть перекодированы в компактное представление (JWS/JWE Compact Serialization): к заголовку и полезной нагрузке применяется алгоритм кодирования Base64-URL, после чего добавляется подпись и все три элемента разделяются точками («.»).



Метод аутентификации *

- Создадим новый endpoint login
- Обязательно POST запрос
- Параметры GET запроса не шифруются

```
        r.POST("/login", a.Login) // там где мы ра

...
type loginReq struct {
    Login    string `json:"login"`
    Password string `json:"password"`
}

type loginResp struct {
    ExpiresIn   int    `json:"expires_in"`
    AccessToken string `json:"access_token"`
    TokenType   string `json:"token_type"`
}

func (a *Application) Login(gCtx *gin.Context) {
        ...
}
```

Метод аутентификации *

- Проверка логина и пароля
- Формирование JWT

```
if req.Login == login && req.Password == password {
        // значит проверка пройдена
        // генерируем ему jwt
        token := jwt.NewWithClaims(cfg.JWT.SigningMethod, &ds.JWTClaims{
                StandardClaims: jwt.StandardClaims{
                        ExpiresAt: time.Now().Add(cfg.JWT.ExpiresIn).Unix(),
                        IssuedAt:  time.Now().Unix(),
                        Issuer:    "bitop-admin",
                },
                UserUUID: uuid.New(), // test uuid
                Scopes:   []string{}, // test data
        })

        if token == nil {
                gCtx.AbortWithError(http.StatusInternalServerError, fmt.Errorf("token is nil"))

                return
        }

        strToken, err := token.SignedString([]byte(cfg.JWT.Token))
        if err != nil {
                gCtx.AbortWithError(http.StatusInternalServerError, fmt.Errorf("cant create str token"))

                return
        }

        gCtx.JSON(http.StatusOK, loginResp{
                ExpiresIn:   cfg.JWT.ExpiresIn,
                AccessToken: strToken,
                TokenType:   "Bearer",
        })
```

Авторизация Golang *

- Авторизацию выносим в middleware
- Она будет применяться ко всем нашим методам - endpoint

```
func (a *Application) WithAuthCheck(gCtx *gin.Context) {
        jwtStr := gCtx.GetHeader("Authorization")
        if !strings.HasPrefix(jwtStr, jwtPrefix) { // если нет префикса то
                gCtx.AbortWithStatus(http.StatusForbidden) // отдаем что не

                return // завершаем обработку
        }

        // отрезаем префикс
        jwtStr = jwtStr[len(jwtPrefix):]

        _, err := jwt.ParseWithClaims(jwtStr, &ds.JWTClaims{}, func(token *
                return []byte(a.config.JWT.Token), nil
        })
        if err != nil {
                gCtx.AbortWithStatus(http.StatusForbidden)
                log.Println(err)

                return
        }
```

Пример 403 Golang

- 403 ответ если пароль и логин не подходят

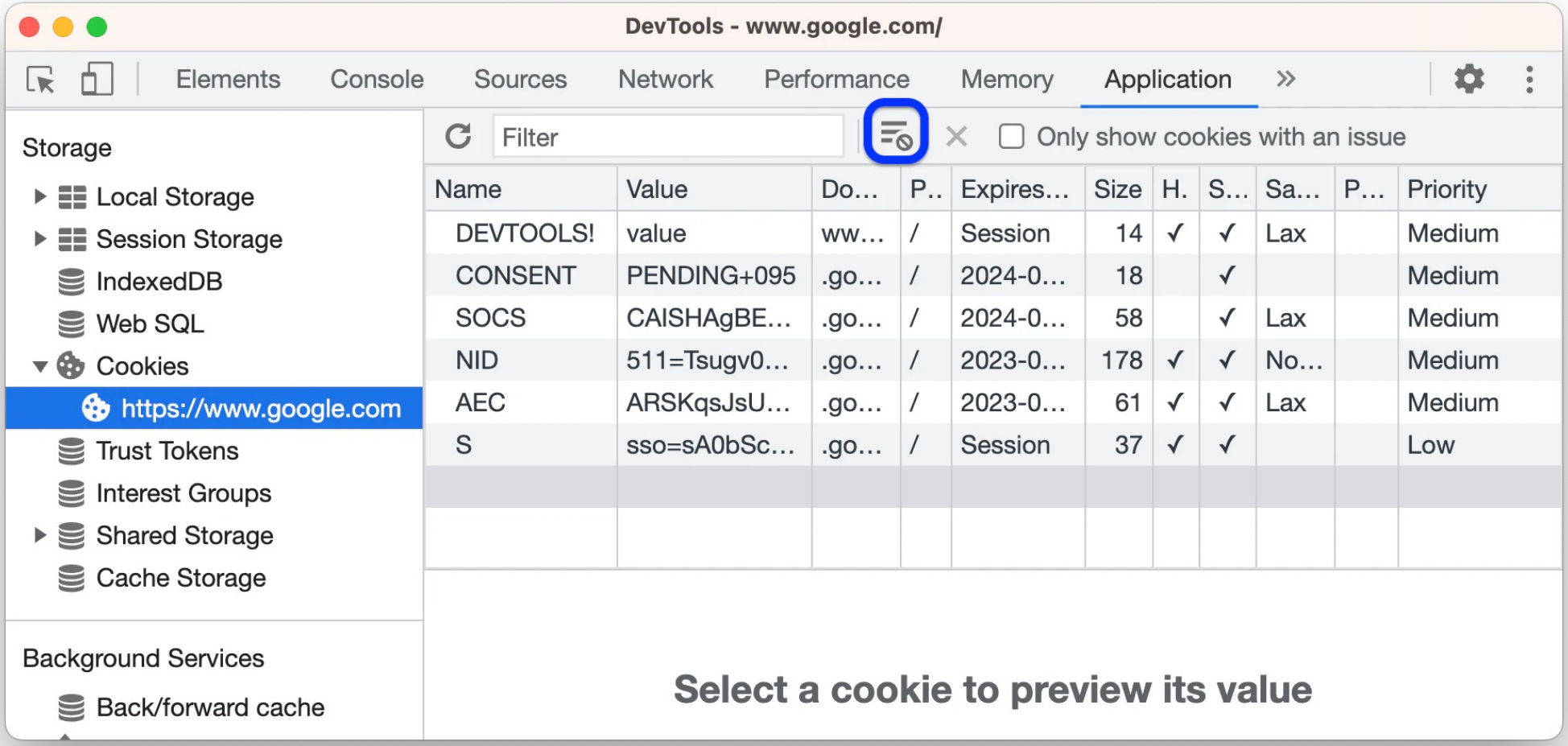
```
$ curl -v --location --request POST 'http://127.0.0.1:8080/login' \
--header 'Content-Type: application/json' \
--data-raw '{
    "login": "login",
    "password": "check1223"
}'
Note: Unnecessary use of -X or --request, POST is already inferred.
*   Trying 127.0.0.1:8080...
* Connected to 127.0.0.1 (127.0.0.1) port 8080 (#0)
> POST /login HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/7.84.0
> Accept: */*
> Content-Type: application/json
> Content-Length: 53
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 403 Forbidden
< Date: Sun, 20 Nov 2022 19:09:16 GMT
< Content-Length: 0
<
* Connection #0 to host 127.0.0.1 left intact
```

Пример 200 Golang

- 200 если логин с правильным логином и паролем

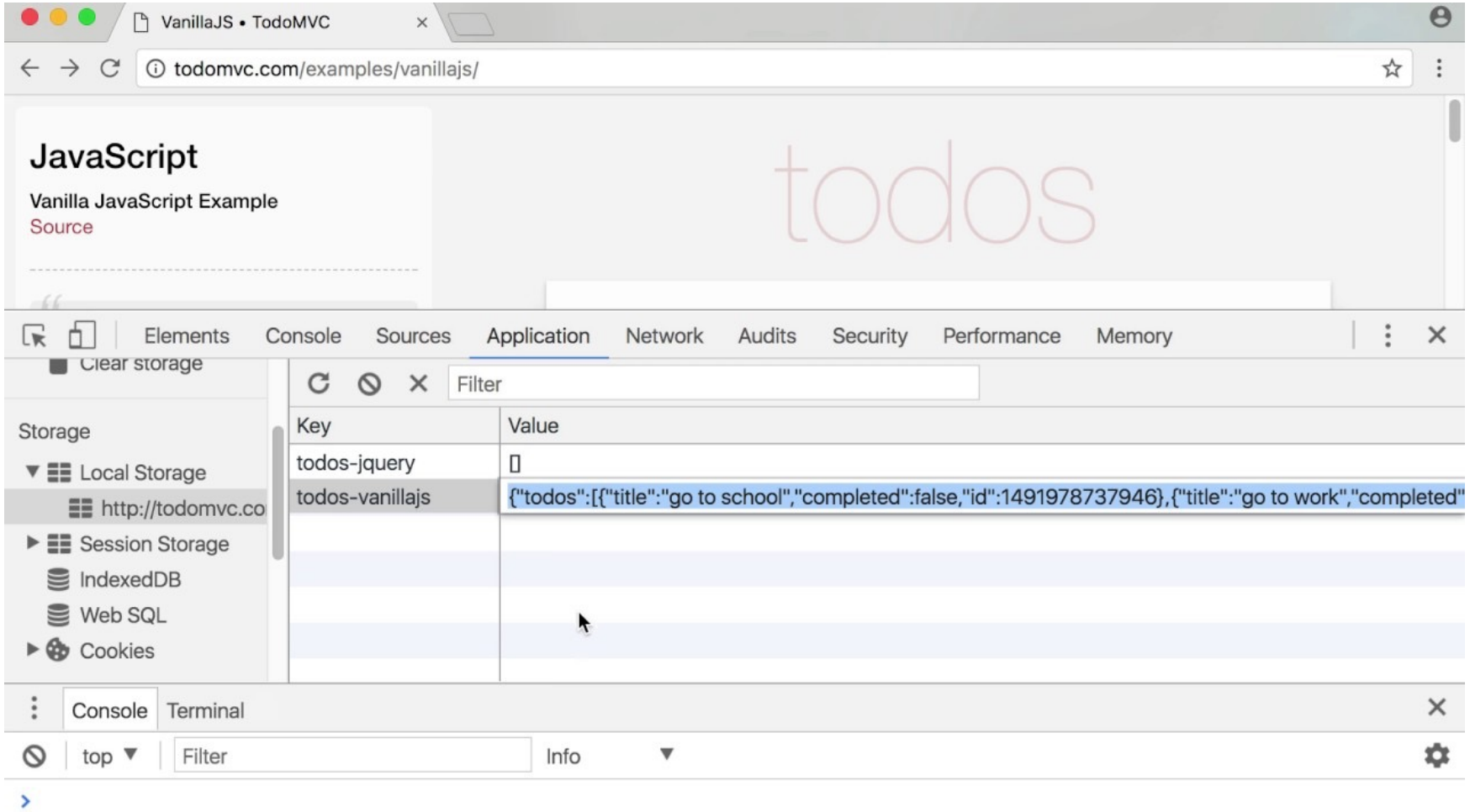
```
$ curl --location --request POST 'http://127.0.0.1:8080/login' \
--header 'Content-Type: application/json' \
--data-raw '{
    "login": "login",
    "password": "check123"
}'
{"expires_in":3600000000000,"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE2Njg5NzUwNz
```

Вкладка Application. Cookie



Вкладка Application. Local Storage

- В варианте с JWT в 7 лабораторной мы используем Local Storage (локальное хранилище) браузера для хранения JWT
- В обоих случаях нас интересует **вкладка Application**

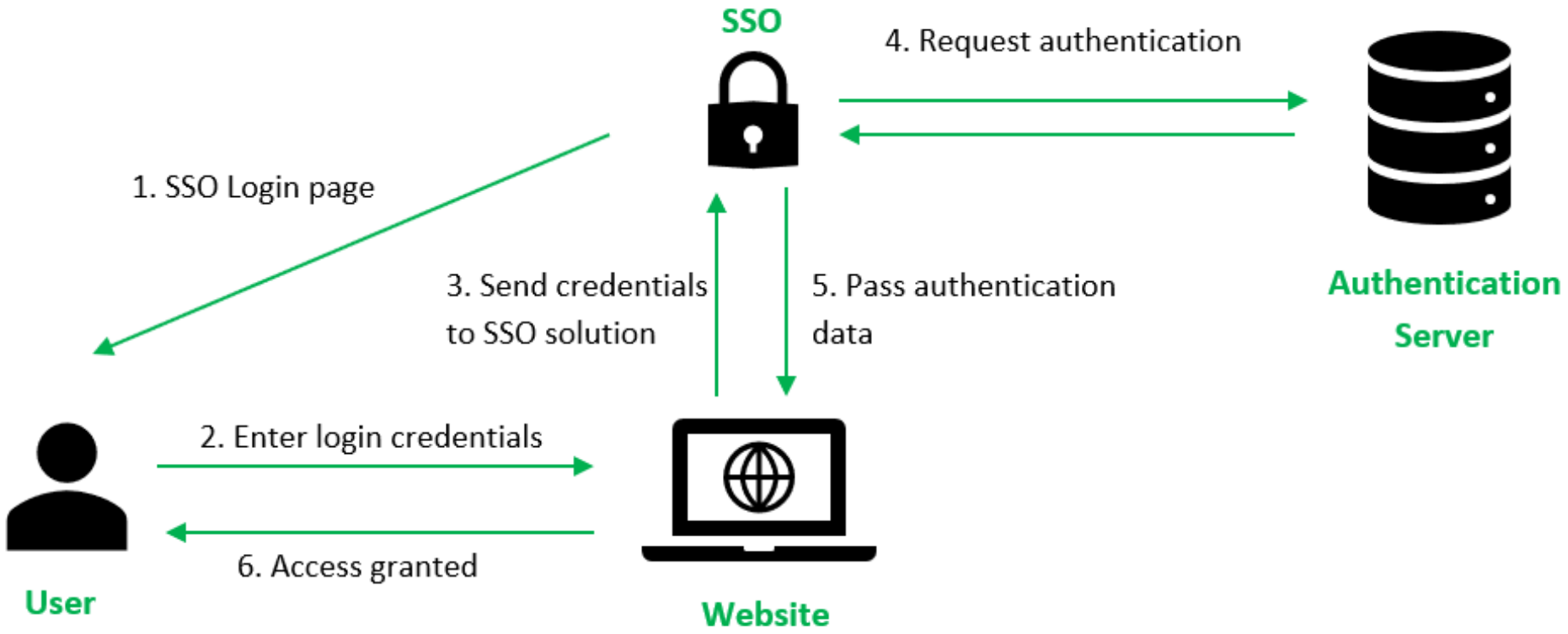


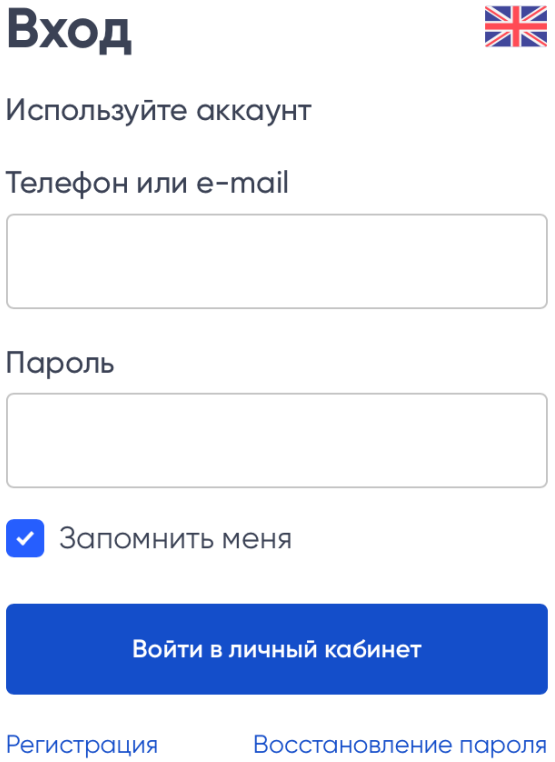
Проблемы *

- Одной из проблем аутентификации и информационной безопасности является то, что пользователи, как правило, используют несколько различных сервисов (например, на Google, Microsoft, Apple и др.), и, соответственно, несколько учётных записей со своими логинами и паролями.
- Таким образом пользователям требуется хранить и защищать множество логинов-паролей.
- Поскольку каждый из сервисов имеет собственную систему безопасности со своими уязвимостями и недостатками, то всё это наносит ущерб удобству и безопасности пользователям
- **Решение** – использование **SSO**

Single Sign-On *

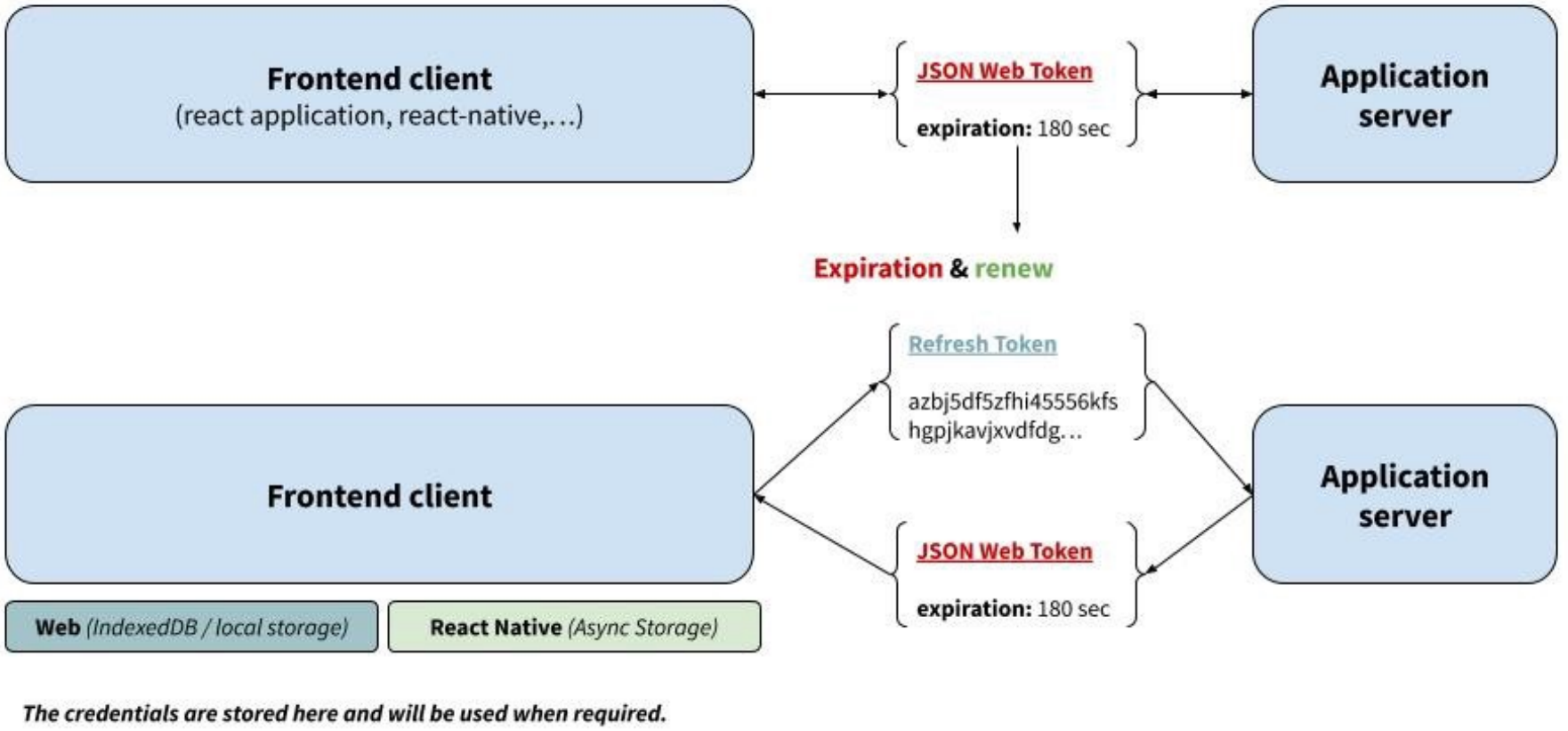
- **Технология единого входа SSO** — технология, при использовании которой пользователь переходит из одного раздела портала в другой, либо из одной системы в другую, не связанную с первой системой, без повторной аутентификации.





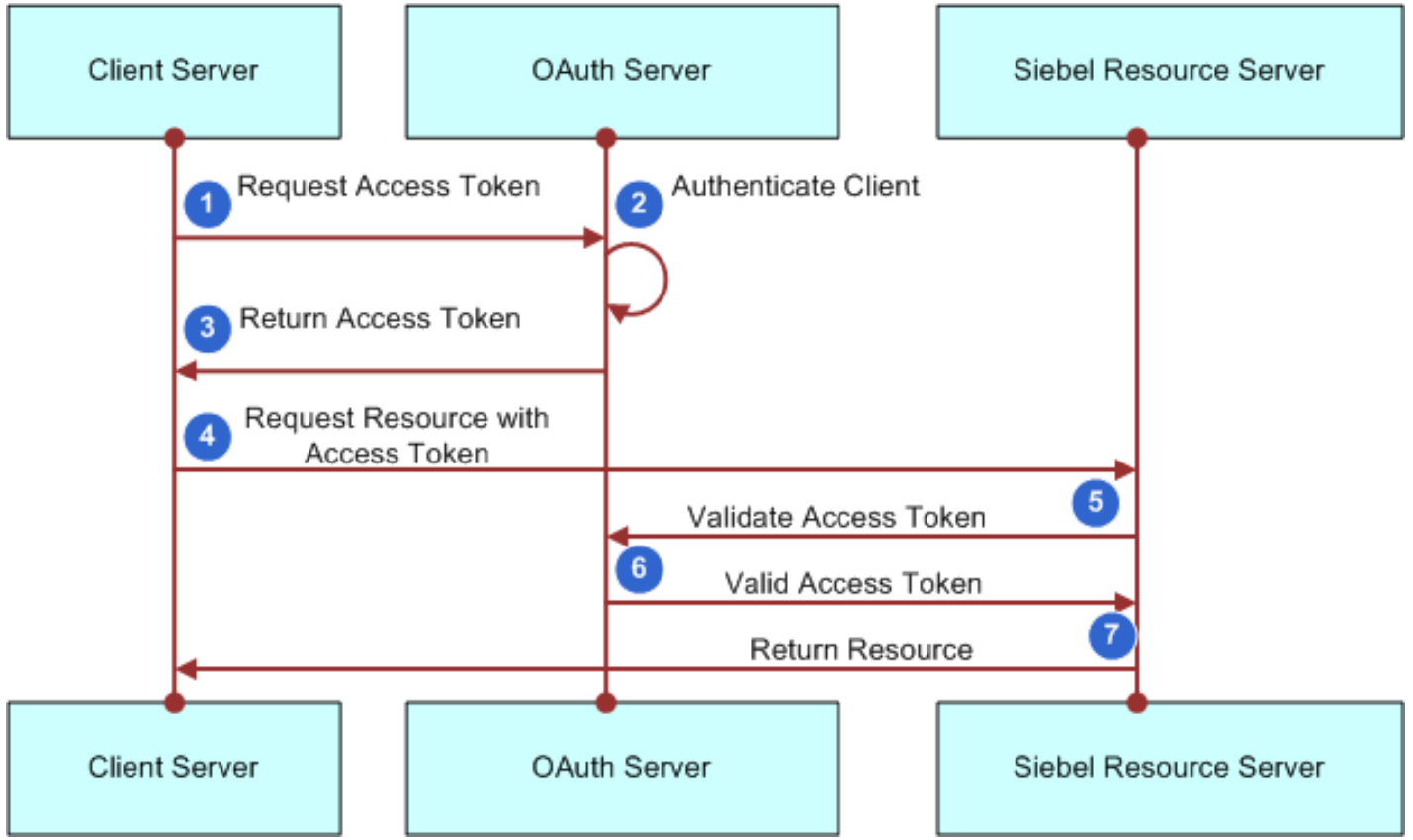
Access и Refresh *

- Давайте усложним схему и сделаем два разных JWT (**OAuth**)
- **Access JWT** используем при авторизации и проверяем во всех наших методах
- **Refresh JWT** выпускаем при аутентификации и каждые 3 минуты отправляем на сервер, чтобы выпустить Access JWT



Oauth *

- **OAuth** — открытый протокол (схема) авторизации, обеспечивающий предоставление третьей стороне ограниченный доступ к защищённым ресурсам пользователя.
- Без передачи ей (третьей стороне) логина и пароля



Двухфакторная аутентификация

- Двухфакторная аутентификация — это метод идентификации пользователя в каком-либо сервисе (как правило, в Интернете) при помощи запроса аутентификационных данных двух разных типов
- Это обеспечивает двухслойную, а значит, более эффективную защиту аккаунта от несанкционированного проникновения.
- На практике это обычно выглядит так: первый рубеж — это логин и пароль, второй — специальный код, приходящий по SMS или электронной почте.

